

Towards a Hybrid Quantum-Classical Computing Framework for Database Optimization Problems in Real Time Setup

Hanwen Liu[§]

*Department of Computer Science
University of Southern California
Los Angeles, CA, USA
hanwen_liu@usc.edu*

Ibrahim Sabek[§]

*Department of Computer Science
University of Southern California
Los Angeles, CA, USA
sabek@usc.edu*

Abstract—Quantum computing has shown promise for solving complex optimization problems in databases, such as join ordering and index selection. Prior work often submits formulated problems directly to black-box quantum or quantum-inspired solvers with the expectation of directly obtaining a good final solution. Due to the black-box nature of these solvers, users cannot perform fine-grained control over the solving procedure to balance the accuracy and efficiency, which in turn limits flexibility in real-time settings where most database problems arise. Moreover, it leads to limited potential for handling large-scale database optimization problems. In this paper, we propose a vision for the first real-time quantum-augmented database system, enabling transparent solutions for database optimization problems. We develop two complementary scalability strategies to address large-scale challenges, overcomplexity, and oversizing that exceed hardware limits. We integrate our approach with an database query optimizer as a preliminary prototype, evaluating on real-world workload, achieving up to 14x improvement over the classical query optimizer. We also achieve both better efficiency and solution quality than a black-box quantum solver.

Index Terms—Quantum computing, Query optimization, Data systems on modern hardware

I. INTRODUCTION

Database systems encompass multiple optimization problems, such as join ordering (e.g., [1]–[5]), transaction scheduling (e.g., [6], [7]), multi-query optimization (e.g., [8], [9]), and index tuning [10]–[12]. The quality of their solutions is critical to overall performance. As data volumes grow exponentially and query workloads become more complex, these optimization tasks have become increasingly difficult to solve efficiently [13]. These problems are inherently combinatorial, and for decades, database systems have relied on heuristic and other classical optimization techniques (e.g., [1], [2], [6], [8]–[11]) that balance solution quality and computational efficiency. While such methods are effective in practice, they become less reliable when the search space is large, highly-complex, or must be explored under real-time or adaptive conditions. In these scenarios, the objective often shifts from

finding the optimal solution to producing a high-quality solution quickly, where accepting a suboptimal result is justified if it substantially reduces optimization time.

A prime example is the join order optimization problem, a key challenge in query optimization that is NP-hard [14] due to the exponential growth of the search space with the number of relations. Dynamic programming (DP)-based approaches (e.g., [15]) can compute optimal results by exhaustively enumerating all possible join sequences for small queries (i.e., queries with relatively few relations) but become impractical for larger ones. To address this limitation, a variety of heuristic methods (e.g., [15]–[18]) have been proposed to balance solution quality and computational efficiency.

More recently, machine learning based techniques have emerged as promising alternatives for database optimization (e.g., [3]–[5], [7], [12], [19]). However, they continue to face significant challenges in hard combinatorial settings, including cold-start issues (e.g., [3]), dependence on large training datasets (e.g., [20]), and the need for frequent retraining as workloads evolve (e.g., [19]).

Fortunately, recent advances in quantum hardware (e.g., [21]–[23]) have positioned quantum computing as a powerful and promising computational paradigm. Grounded in widely accepted complexity theoretic assumptions [24]–[26], quantum systems are believed to surpass classical systems in computational capability, with the potential to (i) achieve quantum advantage [27]–[29] on specific problems by solving them faster than the best known classical algorithms, for example in cryptanalysis [30], [31] and simulation [27], [28], and (ii) efficiently explore exponentially large search spaces, particularly for certain optimization problems [32], [33]. Moreover, quantum computing is becoming increasingly accessible through cloud platforms such as D-Wave [34] and IBM Quantum [22]. Forward looking projections [35], [36] anticipate rapid growth in quantum capacity and the emergence of on-premise hardware in the near future, enabling lower latency execution and tighter system integration.

Despite their transformative potential, quantum computing systems remain in an early stage of development and face

[§]Both authors have equal contributions, and their names are sorted alphabetically.

significant hardware-related challenges. Current devices operate in the noisy intermediate-scale quantum (NISQ) regime and exhibit limitations such as restricted qubit counts, sparse connectivity, substantial noise, and the absence of robust error correction [37]. Quantum annealers [21], such as those developed by D-Wave [34], have shown promise for combinatorial optimization in real-world applications (e.g., production scheduling [32], power network optimization [33]) by exploiting quantum tunneling [38] and parallel exploration of solution spaces. However, their limited qubit capacity and sparse hardware topologies (e.g., Pegasus [39], Zephyr [40]) constrain the size and density of problems that can be efficiently embedded. As a result, pure quantum annealing remains insufficient for solving complex, large-scale database optimization tasks (e.g., [41]–[46]).

To overcome these bottlenecks, hybrid quantum–classical computing has emerged as a practical paradigm in which quantum processing units (QPUs) accelerate only the most computationally intensive subproblems within broader classical optimization pipelines [47], [48]. Such hybridization leverages the complementary strengths of both worlds: classical processors handle large-scale orchestration and problem decomposition, while quantum annealers focus on sampling and exploring complex energy landscapes that are otherwise computationally prohibitive. Yet, applying existing hybrid quantum–classical methods to database optimization introduces several unique challenges, especially in a *real-time* setup:

C1. Solver Opacity: Rely on black-box solvers. Most current research (e.g. [49]–[51]) relies heavily on commercial, black-box quantum solvers (e.g., D-Wave’s CQM Solver [48] and NL-Solver [52]), and on quantum-inspired solvers (e.g., Digital Annealer [53]). Without the required transparency from quantum solvers, researchers typically formulate optimization problems in the Quadratic Unconstrained Binary Optimization (QUBO) format and submit them directly to these solvers (e.g., [54], [55]). This black-box usage restricts fine-grained user control over the quantum annealing process and limits flexibility for debugging and tuning. Consequently, it hinders the design of more efficient algorithms for database optimization in real-time settings.

C2. QUBO Overcomplexity. Most existing approaches faithfully map the original problems into QUBO form [44], [51], [54], [56]–[58]. Although this preserves semantics, it produces overly complex QUBOs that yield dense interaction graphs and excessive auxiliary variables. This complicates variable-to-qubit mapping and slows down embedding on sparsely connected hardware, such as D-Wave’s Pegasus [39] and Zephyr [40]. Moreover, the added complexity degrades solver performance in finding high-quality solutions. Many database problems, however, exhibit redundancy or sparsity. For example, in join order optimization, if joins $A \bowtie B$ and $B \bowtie C$ are selected, there is no need to introduce additional variables to encode that tables A and C are connected [51], [59]. Blindly preserving all correlations increases complexity without commensurate gains.

C3. QUBO Oversize: Beyond hardware limits and

semantics-aware decomposition. As database optimization problems grow, mapping a QUBO to physical qubits can exceed hardware capacity. Feasibility requires that every decision variable be mapped to a physical qubit. Problems beyond this threshold must be decomposed and later recomposed. Generic decomposition that ignores database semantics treats each QUBO as a standalone, independent problem. Missing structural semantic information forfeits potential simplifications during recomposition and limits adaptability when parameters change slightly. Practical scaling, therefore, demands decomposition that preserves task semantics and minimizes shared variables across subproblems, so that recomposition remains efficient and the final solutions remain valid.

To address these challenges, we propose a scalable hybrid quantum-classical framework for solving large-scale database optimization problems **in real-time settings**. Specifically, this framework **simplifies** (i.e., iterative correlation relaxation) or **partitions** (i.e., problem-aware decomposition) QUBO structures while preserving solution correctness. This paradigm directly leverages the lowest-level operations (i.e., sampling) on the quantum annealer, enabling transparent and fine-grained control to achieve a balance between solution quality and runtime efficiency. Our preliminary prototype and evaluation demonstrate up to a 14x improvement over the classical database component (i.e., PostgreSQL’s query optimizer) and a commercial black-box quantum solver.

This paper presents three key takeaways:

- 1) **New Perspective:** We introduce a vision for the *first real-time quantum-augmented database system* that tightly integrates a hybrid quantum and classical optimizer, enabling transparent solutions for large-scale database optimization problems.
- 2) **New Methods:** We develop two complementary scalability strategies. First, an iterative correlation relaxation framework that preserves the full set of variables while pruning and selectively reintroducing correlations based on semantics and their impact on solution quality. Second, a problem-aware decomposition–embedding–sampling–composition pipeline that preserves structure while partitioning QUBOs into semantically meaningful subproblems to solve large-scale optimization problems.
- 3) **New Research Direction and Challenges:** We outline a promising hybrid quantum-classical framework for large-scale optimization that is deeply integrated within database systems, while noting that efficiency, simplicity, and generalizability remain ongoing challenges.

This paper is organized as follows. Section II introduces preliminaries on quantum annealing and essential background on current quantum computing methodology. Section III presents the system architecture and formalize the problem setting. Section IV details the two proposed methods. Section V describes a preliminary prototype integrated with a database query optimizer. Section VI reports preliminary experimental results. Finally, Section VII summarizes the findings and

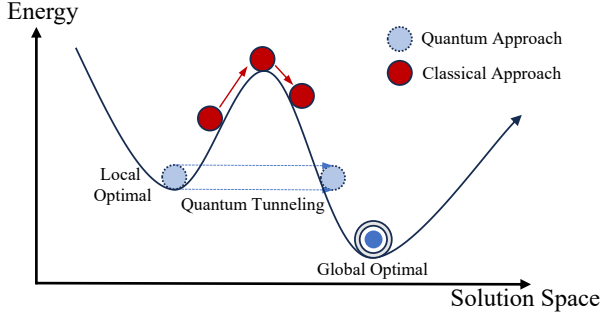


Fig. 1: Quantum Annealing Compare with Classical Approach.

contributions of this vision paper.

II. BACKGROUND & PRELIMINARIES

Quantum annealing (Section II-A) serves as the theoretical foundation and core technology in this work. It underpins the hybrid quantum classical optimization workflow (Section II-B). We then survey correlation relaxation for QUBO (Section II-C) and problem-aware hybrid optimization pipelines (Section II-D).

A. Quantum Annealing

Quantum Annealing (QA) is a computational paradigm utilizing quantum hardware to solve complex optimization problems [60]. It shares similarities with classical simulated annealing (SA) [61]: Both methods explore the search space by proposing random perturbations to the current state. Fig. 1 compares SA and QA. In SA, a proposed move is accepted if it lowers the objective; otherwise, it is accepted with a probability that depends on the current temperature. Exploration in SA is driven by thermal fluctuations under a temperature schedule. In QA, the system evolves under a time-dependent Hamiltonian; the transverse field induces quantum fluctuations that promote exploration. Quantum tunneling further enables the system to traverse energy barriers rather than surmount them thermally, allowing escape from suboptimal states (i.e., avoid becoming trapped in local minima), thereby increasing the likelihood of converging to a global optimum.

QA solves problems that can be formulated as Quadratic Unconstrained Binary Optimization (QUBO) [62]. In the QUBO form, QA seeks a binary assignment $s \in \{0, 1\}^n$ that minimizes the quadratic energy function [63]:

$$E(\vec{s}) = \sum_{ij} J_{ij} s_i s_j + \sum_i h_i s_i,$$

where s_i and s_j are binary variables representing the qubit states, h_i denotes the local bias on qubit i , and J_{ij} represents the interaction strength between qubits. During quantum annealing, the initial superposition evolves toward a state that minimizes the system's energy, encoding the solution to the original combinatorial optimization problem.

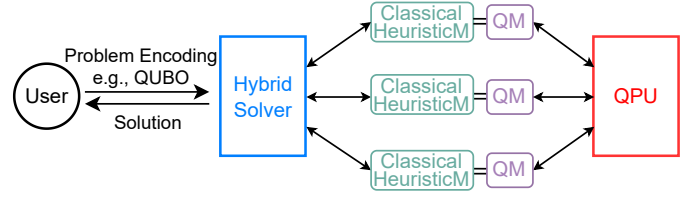


Fig. 2: A Generic Workflow for Hybrid Quantum-Classical Optimization.

B. Hybrid Quantum-Classical Optimization

Hybrid quantum-classical optimization couples classical heuristics with quantum annealing (QA) [60] to improve scalability and solution quality. As shown in Fig. 2, the user encodes the optimization problem and submits it to a hybrid solver that launches parallel workers, each combining a classical heuristic module with a quantum module (QM). The heuristic module preprocesses, decomposes, and refines the search space, while the QM submits corresponding quantum queries to the Quantum Process Unit (QPU) and returns samples with energies; this feedback guides subsequent heuristic updates. The loop continues until a time budget or stopping criterion is met, after which the best solution is returned. Most commercial, black-box quantum solvers, such as D-Wave's CQM-Solver [48] and NL-Solver [52], follow this hybrid workflow.

C. Correlation Relaxation for QUBO

Correlation relaxation simplifies the original QUBO while retaining the full set of decision variables. This technique has been explored in several hybrid optimization methods [34], [64], [65]. A common strategy is to prioritize correlations with the largest energy contribution; in a QUBO representation, this amounts to ranking the pairwise terms in J by their impact on the Hamiltonian energy [66]. However, a purely energy-based criterion does not carefully account for semantic correctness in database optimization problems, where correlations with lower energy may enforce critical semantics. For example, in join order optimization [46], [54], a low energy correlation may encode a validity constraint that prevents duplicate joins or infeasible join structures, and pruning this correlation can yield an invalid plan. Similarly, in index tuning [67], some low-weight constraints ensure that storage budgets are respected, and removing them can produce infeasible solutions. Therefore, energy-only correlation pruning, which does not consider semantic meaning, is insufficient for database optimization.

D. Problem-aware Hybrid Optimization Pipelines

Commercial quantum providers (e.g., D-Wave [34], IBM [68], Fujitsu [69]) offer hybrid optimization pipelines. Although they permit limited user customization, these pipelines rely on generic techniques, such as minor embedding [70] and unconstrained merging [34], which do not account for problem-specific semantics, thereby limiting their effectiveness for database optimization use cases. To be effective in

practice, decomposition must leverage problem-specific characteristics rather than generic QUBO graph properties, minimizing shared variables across subproblems to avoid recomposition bottlenecks. Few prior quantum-classical database optimization efforts [55], [57], [71] have incorporated problem characteristics only at the logical level (e.g., pruning unlikely join trees [55] or low-contention transactions [57]) prior to QUBO construction. While this reduces size, it treats the QUBO structure itself as a black box, failing to account for how database semantics map to embeddings and solver configurations. As a result, these solutions are not reusable or adaptable to workload shifts. For example, aggressive pruning of join trees might exclude query plans that could later become optimal under changing workloads.

III. SYSTEM OVERVIEW

In this section, we first present an overview of our vision for a quantum-augmented database system and its key components (Section III-A). We then formally define the problem setting for achieving the best solution accuracy under a given time constraint, which guides the application of our system (Section III-B).

A. System Architecture

Figure 3 presents an overview of our vision for quantum-augmented database systems. We design a hybrid quantum-classical database optimizer that interfaces closely with core DBMS components, including the transaction manager, the query optimizer, and the execution engine. This visionary system resolves the three challenges in solving the large-scale database optimization problem (Check Section I).

In the workflow of the hybrid quantum-classical DB optimizer, the system first receives a large-scale database optimization problem, such as transaction scheduling, query or operator scheduling, or the join order problem. The problem is then encoded into a variable domain model, e.g., a QUBO. If the QUBO contains more variables than the capacity of the quantum annealer (Challenge C3), the system applies problem-aware decomposition (details in Section IV-B). Afterward, the variables are mapped to physical quantum qubits as the initial step for execution on quantum hardware. If the QUBO is overly dense (Challenge C2), correlation relaxation (details in Section IV-A) is applied. The quantum sampling, which is the lowest-level and fundamental operation, is then executed on the quantum hardware. Based on solution quality and user-defined timing constraints, our system determines whether to invoke the iterative correlation relaxation mechanism (details in Section IV-A) to further improve solution quality. If so, the obtained samples and their energies are used to update the QUBO parameters, and the process continues. Otherwise, the reverse of decomposition, problem-aware composition (details in Section IV-B) is performed, to produce the final QUBO solution. Finally, it is then decoded to obtain the solution to the original problem. This end-to-end workflow does not rely on any black-box quantum solver (Challenge C1) and provides

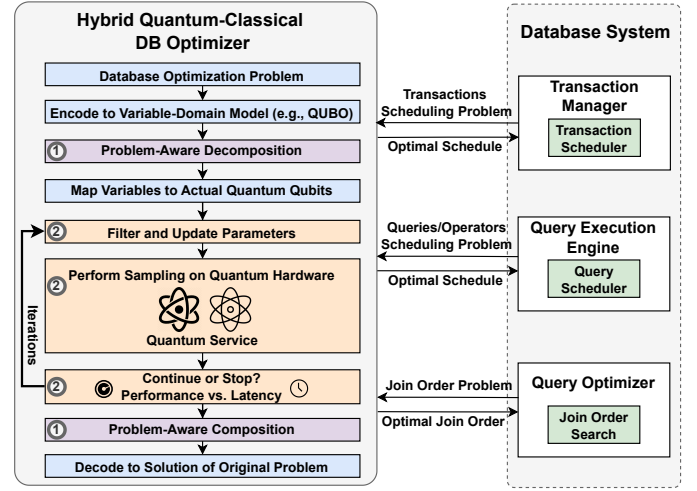


Fig. 3: Overview of a Quantum-Augmented Database System.

transparent control over quantum sampling for solving large-scale optimization problems on quantum hardware.

B. Problem Statement

We consider optimization tasks for executing a query Q over a database $\mathcal{D}$. A candidate solution, denoted $s(z)$ (e.g., a query plan), is represented by an encoding $z \in \{0,1\}^n$ in the quantum solution space, where n is the number of binary decision variables in the encoding. The objective is to maximize solution quality while adhering to a strict budget for quantum processing time.

Performance. We measure solution quality by the performance of executing query Q on database $\mathcal{D}$. Specifically,

$$\max \text{Perf}(s; Q; \mathcal{D}) \iff \min \text{Latency}(s; Q; \mathcal{D}),$$

where $\text{Latency}(s; Q; \mathcal{D})$ is the execution time of Q under solution s on $\mathcal{D}$, so a lower latency indicates a higher quality.

Quantum Lifecycle. This quantity represents the time consumed by our quantum computing pipeline to obtain the candidate solution z . To reach a high-quality solution, the algorithm may interact with the quantum hardware multiple times (see Section IV-A). Let N be the number of iterations. In iteration i , let T_{Q_i} denote the quantum solver compute time, T_{C_i} the communication time between the DBMS and the quantum service, and T_{R_i} the client-side QUBO refinement time. The total quantum lifecycle time is

$$\text{Time}_{\text{Quantum}}(z) = \sum_{i=1}^N (T_{Q_i} + T_{C_i} + T_{R_i}),$$

where $\text{Time}_{\text{Quantum}}(z)$ is the end-to-end time spent within the quantum lifecycle to produce z .

Performance Maximization under a Time Budget. Given a time budget $\tau > 0$, our goal is to find, within this budget, the

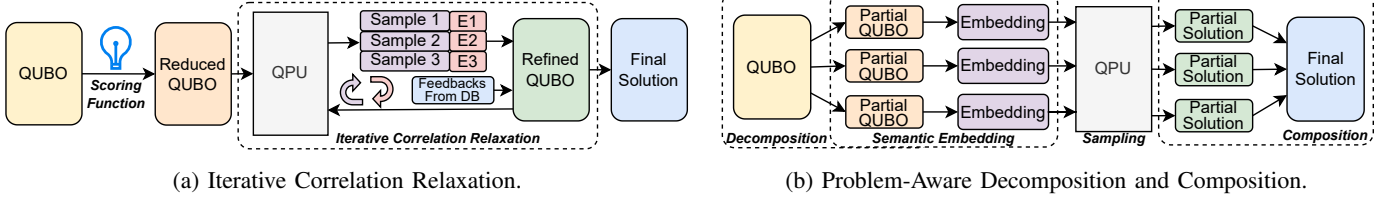


Fig. 4: Scalable Hybrid Quantum-Classical Methods.

solution that yields the lowest query execution latency. The problem is formulated as

Given: $\mathcal{Q}, \mathcal{D}, \tau \in \mathbb{R}_{>0}$

Decisions: $z \in \{0, 1\}^n, s = s(z)$

Minimize: $\text{Latency}(s; \mathcal{Q}; \mathcal{D})$

s.t. $\text{Time}_{\text{Quantum}}(z) \leq \tau$ (Time budget)
 $g_j(s) \leq 0, j = 1, \dots, m$ (Original constraints)

IV. SCALABLE HYBRID QUANTUM-CLASSICAL METHODS FOR LARGE-SCALE DATABASE OPTIMIZATION

In this section, we present two approaches. Both address the first challenge (C1) by avoiding reliance on black box solvers and by providing transparent control over the quantum hardware. The first approach addresses the second challenge (C2) of QUBO overcomplexity, enabling a tunable trade-off between efficiency and accuracy. It introduces an iterative correlation relaxation framework that retains the full set of variables while pruning and progressively reintroducing correlations to simplify QUBOs (Section IV-A). The second approach addresses the third challenge (C3), where the QUBO exceeds current hardware capacity. It adopts a problem-aware decomposition and composition strategy that preserves both variables and correlations, partitions the QUBO into smaller, semantically meaningful subproblems, solves them independently, and then reconciles the results (Section IV-B).

A. Iterative Correlation Relaxation

Selectively relaxing correlations between variables yields a sparser interaction structure, which simplifies mapping correlation onto the quantum annealer for overly complex QUBO instances and naturally enables more efficient execution. With proper correlation relaxation, even though the reduced QUBO omits several correlations, it can still produce solutions of comparable quality. In this case, we propose an iterative correlation relaxation framework to address these limitations by balancing QUBO sparsity (i.e., over-relaxation leads to a sparse QUBO) and solution accuracy (i.e., limited relaxation maximally maintains the original QUBO features) through problem-specific heuristics and feedback-driven refinement. This iterative mechanism also provides flexibility in controlling the trade-off between the solution accuracy and latency from quantum computing.

Fig. 4a shows a complete framework. We first obtain a reduced QUBO that preserves only the correlations with the

largest estimated influence from the initial QUBO. These correlations are selected using scoring functions tailored to each database problem. For example, in join order optimization, we prioritize correlations between joins that yield large intermediate result cardinalities [72]. For transaction scheduling, we retain edges that involve transactions with frequent contention [73]. For index tuning, we emphasize access pattern correlations among indexes that are often co-accessed across queries [74]. In these cases, the scoring functions yield higher scores for these correlations, which serve crucial roles in the original database optimization problem.

The QPU, e.g., the D-Wave quantum annealer [34], is used as a parameterized sampler [75] that produces candidate solutions (i.e., samples and corresponding energies) under the reduced correlation set with the current coefficients h and J (introduced in Section II-A). The empirical distribution of these samples provides the first source of feedback. Additionally, closely integrated with the actual database system, this provides more feedback, such as solution quality metrics (e.g., current query plan cost in the join order problem) and constraint violation statistics (e.g., the number of invalid join subplans). These signals are analyzed to identify missing correlations that have a significant impact on actual query execution performance. The identified correlations, i.e., the pairwise terms in J , are selectively reintroduced in subsequent iterations, thereby progressively refining the QUBO structure.

The iterative procedure terminates when any of the following conditions is met: (i) solution quality stabilizes, e.g., no further cost reduction over several consecutive iterations; (ii) the energy landscape reaches a predefined stability threshold; or (iii) the maximum number of iterations is reached.

Convergence Strategies. In iterative QUBO refinement, a central challenge is to devise an effective strategy that ensures convergence. Simply reintroducing correlations in a greedy manner may lead to oscillations or premature convergence, especially when the database optimization problem exhibits a dense cost structure. To address this challenge, we minimize the Kullback-Leibler (KL) divergence [76] between the ideal Boltzmann distribution [75], [77] over valid solutions and the empirical distribution generated by the QPU. Aligning these distributions approximates the energy landscape of the fully constrained problem. The resulting iterative updates resemble machine learning based gradient descent, progressively reducing the discrepancy between the two distributions. As complementary options, entropy-based regularization [78] can serve as fallbacks when KL-divergence minimization exhibits

instability on specific workloads.

B. Problem-Aware Decomposition and Composition

To address the third challenge, where the size of a QUBO exceeds quantum hardware capacity, we preserve the full problem structure by retaining all variables and pairwise interactions, while decomposing the problem into smaller subproblems that are solved individually and then recomposed (e.g., [34], [57], [68], [69], [79]–[81]). We propose a problem-aware hybrid optimization pipeline that fully considers the key semantics of the underlying database optimization problem. We focus on join order optimization [51], [54], [63] and transaction scheduling [6], [56] as representative use cases. The pipeline is generalizable to other problems. It comprises four stages, detailed as follows.

1) **Decomposition Stage:** In this stage, large QUBOs are divided into smaller subproblems to reduce inter-partition dependencies while preserving all correlations relevant to database operation semantics. For join order optimization, we leverage the join graph representation introduced in [59], which encodes candidate binary join operations as nodes (i.e., QUBO variables) and uses edges (i.e., QUBO correlations) to indicate the presence of shared base relations. Each spanning tree of this graph corresponds to a valid join plan, and the weights on edges and vertices capture join costs and selectivity factors. Based on this model, we design decomposition heuristics that minimize the number of shared join keys across partitions, thereby enabling partial plans to be solved independently while retaining cost accuracy.

For transaction scheduling, we partition according to contention profiles [82], grouping transactions that frequently contend and isolating independent ones to enable parallel solving while preserving correctness. A central challenge in both settings is selecting the right decomposition granularity. Partitions that are too coarse may exceed hardware capacity, whereas partitions that are too fine increase the complexity of future composition. We address this by developing adaptive decomposition strategies guided by workload statistics, such as join selectivity and transaction contention levels, to balance subproblem size and future composition cost.

2) **Embedding Stage:** In this stage, we map the variables of each decomposed subproblem onto the QPU while preserving the semantic locality of the underlying database operation. This is unlike generic minor embedding methods [70], which often produce long and fragile chains. Chains group multiple physical qubits into a superqubit to provide stronger effective connectivity than couplings between single qubits. However, chains are fragile in practical quantum execution, which makes them ill-suited and unreliable for representing the logical relationships inherent in the problem. For join order optimization, we embed all binary join operations that involve a given base table to reflect their shared data paths. For transaction scheduling, we map critical conflict pairs to low-noise regions of the device to reduce chain breaks. We maintain a library of reusable embedding templates (e.g., for cyclic join patterns [46], [54]), which can be adapted to

different hardware architectures, such as D-Wave Pegasus [39] and Zephyr [40].

3) **Sampling Stage:** In this stage, we apply QPU sampling strategically, adapting to the characteristics of each decomposed subproblem in order to obtain high-quality solutions. For example, for subproblems in join ordering that involve many relations or large intermediate results, we allocate longer annealing time or adopt adaptive annealing schedules to allow the QPU to explore the solution space more thoroughly. For low complexity or less critical subproblems, we use shorter runs to improve efficiency. Current quantum hardware often uses static parameters for all QPU tasks, which leads to inefficiency when the subproblem complexity varies [34], [69]. This strategy employs adaptive sampling policies that tune annealing time, the number of reads, and bias schedules based on subproblem cost estimates and solver feedback (e.g., energy variance). By allocating resources according to subproblem difficulty and updating parameters from feedback, the sampling strategy makes better use of existing quantum hardware and improves performance while adaptively handling large QUBOs.

4) **Composition Stage:** In this stage, we employ a two-phase coordination framework to reconcile partial solutions into a globally consistent result. In the first phase, we apply domain-aware belief propagation, inspired by prior work on structured Ising models and domain decomposition [66], [75], [77]. These techniques pass messages over overlapping variables to reconcile local optima and steer convergence toward a coherent global solution. For join order optimization, for instance, partitions that share join keys exchange consensus updates to align cost estimates. In the second phase, we apply post-processing heuristics such as very large neighborhood search [83] and tabu search [81] to iteratively refine the merged solution, particularly for transaction scheduling workloads where reconciliation must respect global constraints, including lock ordering and latency budgets. These methods serve as fallback procedures when belief propagation is ineffective or computationally expensive, and they enable incremental improvements without requiring a full re-optimization.

V. PRELIMINARY PROTOTYPE: QUANTUM-AUGMENTED QUERY OPTIMIZER

In our preliminary prototype, we integrate the iterative correlation relaxation approach (Section IV-A) with the PostgreSQL query optimizer [84]. Fig. 5 illustrates the framework of the iterative quantum augmented query optimizer, referred to as *Iter-Q²O*, which serves as the initial prototype for tightly coupling our proposed approaches with the database system.

Upon receiving a user query, *Iter-Q²O* first retrieves cardinality and selectivity statistics from the DBMS engine. These statistics are used to construct the QUBO formulation, which parameterizes both the objective (e.g., cost function) and the constraints (e.g., join tree structure). The expected output, the join order sequence, is encoded in the decision variables within the QUBO. Once the QUBO is constructed, denoted *Initial QUBO*, it is submitted to the QPU (i.e., a quantum annealer in our initial prototype). The QPU performs quantum

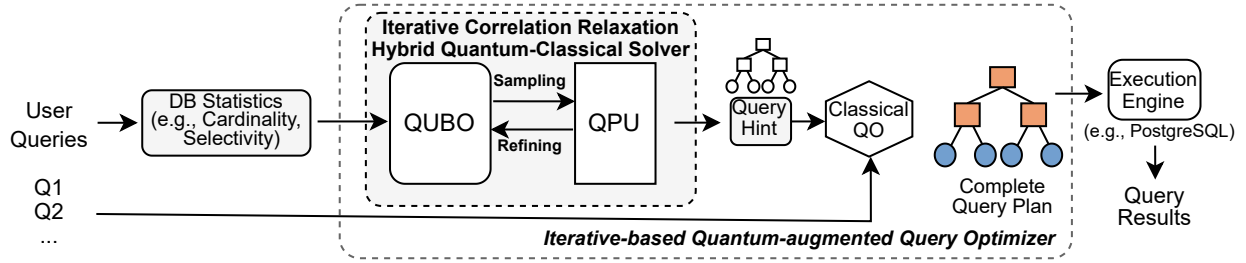


Fig. 5: Framework of Iterative-based Quantum-augmented Query Optimizer.

sampling and returns n candidate solutions, along with their corresponding energies. The distribution of these n samples and their energies is then used to refine Initial QUBO into Refined QUBO according to the methods proposed in Section IV-A. The combined sampling and refinement step is treated as one iteration. The available time budget determines the number of iterations, which may be specified directly by the user or determined automatically by the system. After the prescribed iterations, the iterative-based hybrid quantum-classical solver returns a join order sequence that is the best found under the timing constraint.

This sequence is then translated into a query plan hint, which is used to guide a classical query optimizer in producing a complete query plan.¹ In our preliminary prototype, we use *pg_hint_plan* [90] to tweak PostgreSQL execution plans by embedding hints in SQL comments. The join order is specified with a clause beginning with *Leading*; inner parentheses denote grouping precedence in the join order sequence. For example, the bushy join hint *Leading((a(b c))(d e))* indicates that $b \bowtie c$ is performed first, followed by $a \bowtie (b \bowtie c)$, then $d \bowtie e$, and finally the two intermediate results are joined. PostgreSQL then selects the join operators (e.g., hash join, nested loop join) for each join. Once the optimizer accepts the hint, it is converted into a complete query plan, which the execution engine runs to produce and return the query results.

VI. PRELIMINARY EXPERIMENTS

In this section, we evaluate our preliminary prototype (introduced in Section V), which applies the iterative correlation relaxation approach (introduced in Section IV-A) to solve the query optimization problem in a real-time database system. We first present our experimental setup in Section VI-A. We then evaluate our approach by addressing the following questions: (1) How effective is *Iter-Q²O* for query optimization, as measured by query execution latency, compared with a classical query optimizer? (2) How does *Iter-Q²O* compare with a black-box quantum solver in terms of efficiency and solution quality? (3) What is the end-to-end latency of the approach, and does it maintain an advantage when communicating with an external quantum service? (4) What is the detailed timing

breakdown on the quantum hardware during the core quantum computing process?

A. Experimental Setup

We run all our experiments on a single server equipped with an Intel Core i9-9980XE CPU and 128 GB of RAM. We employ the Advantage2_system1.7 as the quantum sampler on the D-Wave Leap quantum service platform. We set up PostgreSQL 16.4 with the corresponding *pg_hint_plan* version. We utilize D-Wave’s CQM-Solver [48], a representative black-box quantum solver, as a baseline for black box approaches. We evaluate our approach using the *Join Order Benchmark (JOB)* [91] and the *Cardinality Estimation Benchmark (CEB)* [92]. Both JOB and CEB are constructed on the IMDB movie database schema and data, with CEB containing more complex query patterns and a wider range of cardinality estimation challenges than JOB. This complexity makes JOB and CEB suitable and widely adopted benchmarks for testing join order optimization problems.

In the following experiment, when testing the preliminary iterative-based quantum-augmented query optimizer introduced in Section V, we set the number of QUBO sampling-and-refinement iterations to two to invoke the iterative correlation relaxation. In this case, the complete lifecycle (in our 2-iteration setup) comprises: (i) generating the problem from query-related statistics; (ii) connecting to the quantum solver; (iii) submitting the problem and retrieving results; (iv) performing local QUBO refinement; and (v) resubmitting the problem and retrieving the final results.

B. Experimental Results

Query Plan Quality. We evaluate the query plan quality using the actual execution time of JOB and CEB queries on PostgreSQL. Fig. 6 compares the latencies of plans generated by our method with those produced by PostgreSQL. On the JOB workload (113 queries), our approach achieves speedups on 36 queries, with a maximum speedup of 14.02x and an average speedup of 2.70x. CEB queries are used to investigate *Iter-Q²O*’s capability of handling complex workload. On the CEB workload (40 queries), our approach achieves speedups on 13 queries, with a maximum speedup of 5.35x and an average speedup of 2.78x. For the remaining queries, our approach showed only limited performance degradation.

¹This method is commonly used in many well-known query optimizers (e.g., PostgreSQL [85], MySQL [86], Oracle [87]) and learned query optimizers (LQOs) (e.g., [88], [89]).

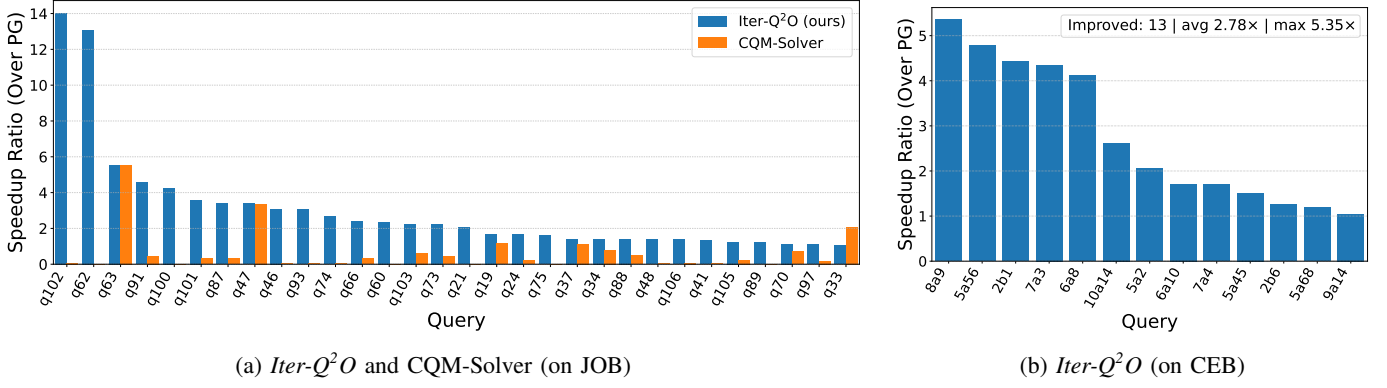


Fig. 6: A Selection of Query Plans Generated by Our Approach Outperforms PostgreSQL’s Default Plans (Iteration = 2).

TABLE I: End-To-End Latency Breakdown and Comparison Between Our Approach and PostgreSQL.

	Workload	JOB		CEB
	Time (ms)	q102	q62	8a9
PG	Planning	3172.32	17.33	82.33
	Execution	8039.26	5177.80	5992.39
CQM	Planning with hints	2467.99	4.54	43.89
	Execution	44968.41	311489.41	512241.75
	Quantum Lifecycle	9092.41	7591.69	9002.50
<i>Iter-Q²O</i> (ours)	Planning with hints	2548.51	7.31	65.91
	Execution	573.27	395.74	1119.09
	Quantum Lifecycle	3168.31	1553.35	1772.95
Our Gain (Over PG)	Absolute Time Saving	4921.49	3238.73	2957.95
	Exec Time ↑	14.02x	13.08x	5.35x
	E2E Latency ↑	1.78x	2.65x	2.05x

Comparison with Black-box Quantum Solver. For a fair comparison, we apply the same initial QUBO formulation to both our prototype and D-Wave’s CQM-Solver. Fig. 6a reports the speedup over PostgreSQL for both methods. For most queries, our approach outperforms the black-box solver. The better performance arises because our iterative solver focuses on the correlations that matter and iteratively corrects the relaxation by reintroducing important correlations. In contrast, we cannot provide problem-aware, fine-grained controls with the CQM-Solver, as it operates as a black box.

Moreover, Table I illustrates that the CQM-Solver incurs a much larger overhead during the quantum lifecycle. This is due to the CQM-Solver enforcing a 5-second minimum time limit before returning results. In contrast, our approach relies solely on the lowest-level operation, quantum sampling, which enables us to circumvent the constraints imposed by black-box quantum solvers.

End-to-End (E2E) Latency. We evaluate End-to-End latency to compare the complete query execution pipeline. For PostgreSQL, E2E latency consists of planning time and execution time. For our approach, E2E latency includes PostgreSQL planning time (with query hint generated by our approach), PostgreSQL execution time, and the time consumed by the complete quantum lifecycle. Table I reports a detailed breakdown of these components for two JOB queries and one

TABLE II: Timing Breakdown on Quantum Service (ms).

Metric	Iter1	Iter2	Total	Avg
Ingress (created→received)	87.896	77.505	165.401	82.701
Solve (received→solved)	361.214	246.652	607.866	303.933
Egress (solved→resolved)	122.164	122.176	244.340	122.170
End-to-end (created→resolved)	571.274	446.333	1017.607	508.803
QPU programming	15.760	15.760	31.520	15.760
QPU sampling	26.356	26.356	52.712	26.356
QPU access time (reported)	42.116	42.116	84.232	42.116
QPU access overhead	0.822	0.798	1.620	0.810
Post-process overhead	0.037	0.036	0.073	0.036

CEB query. Although our approach incurs additional quantum overhead, primarily due to cloud service communication, it generates better join order sequences than PostgreSQL’s defaults, resulting in query plans that reduce execution time and offset this overhead. All reported results are obtained on a local IMDB dataset. As data size increases, the absolute execution time improvement is expected to scale, while the quantum overhead will not scale correspondingly. Overall, our method achieves E2E latency reductions on the order of seconds (i.e., 2957.95 ms, 3238.73 ms and 4921.49 ms), yielding 1.78x to 2.65x E2E latency speedups.

Detailed Timing Breakdown of the Quantum Lifecycle. In Table I, the JOB query q102 spent 3168.31 ms in the quantum lifecycle. We further quantify the portion directly correlated to the quantum hardware itself.

Table II provides a detailed timing breakdown for two iterations of interaction with the quantum annealer. Across the two iterations, the end-to-end times are 571.274 ms and 446.333 ms (average 508.803 ms). This end-to-end time includes all time consumed on the commercial quantum cloud platform from when the problem is created (i.e., *created*) until the solution is returned from the quantum hardware to the quantum cloud platform (i.e., *resolved*). The end-to-end quantum solving time of about 500 ms is encouraging. We also observe that the total time of these two iterations consumed on the quantum cloud platform is 1017.607 ms, which accounts for only a small portion of the client-side observed 3168.31 ms (32.12%). The remaining time is mostly due to multiple bi-directional cloud communications between our database platform and the

commercial cloud platform.

On the quantum hardware itself, the average QPU programming and sampling times are 15.760 ms and 26.356 ms, respectively. The reported QPU access time is 42.116 ms, which represents the time consumed on the core quantum computing process. The QPU access overhead and post-processing overhead are relatively small. Overall, these measurements indicate the current status of solving time and overhead on the quantum hardware. Although most time is spent on cloud communication, the observed end-to-end latency of the quantum hardware and the complete quantum lifecycle is practical for solving real-world and even real-time database problems.

Challenges and Future Improvement. In each iteration of our proposed iterative framework, we refine the QUBO locally based on the obtained samples and energies, and then submit the refined QUBO. Each retrieval and submission incurs non-trivial cloud communication, as shown in Section VI-B, with around 60% of the quantum lifecycle time spent on communication. To reduce this overhead, we propose moving the refinement loop to the server side, for example, by compiling our proposed framework and deploying it as close as possible to the quantum hardware, which would substantially improve runtime efficiency and make the approach better suited to real-world and real-time problem solving.

VII. CONCLUSION

In this paper, we propose the first real-time quantum-augmented database system that tightly integrates a hybrid quantum-classical optimizer. This system enables transparent solutions rather than solely relying on black-box quantum solvers. We then present two scalability strategies targeting large-scale challenges. To address the QUBO overcomplexity challenge, we introduce an iterative correlation relaxation framework that simplifies the QUBO structure while balancing accuracy and efficiency. To address the QUBO oversizing challenge, we propose a problem-aware decomposition and composition pipeline that leverages database semantics to partition QUBOs into subproblems, thereby enabling the solution of large-scale optimization problems. Our preliminary prototype and evaluations demonstrate improved performance over both classical database components and a black-box quantum solver. Moving forward, we aim to design more efficient and accurate quantum augmented systems for large-scale database optimization that are practical in real-time settings.

REFERENCES

- [1] Y. E. Ioannidis, "Query optimization," *ACM Comput. Surv.*, vol. 28, no. 1, p. 121–123, Mar. 1996. [Online]. Available: <https://doi.org/10.1145/234313.234367>
- [2] J. Zhao, K. Su, Y. Yang, X. Yu, P. Koutris, and H. Zhang, "Debunking the myth of join ordering: Toward robust sql analytics," *Proc. ACM Manag. Data*, vol. 3, no. 3, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3725283>
- [3] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making Learned Query Optimization Practical," in *SIGMOD*, 2021.
- [4] X. Yu, G. Li, C. Chai, and N. Tang, "Reinforcement learning with tree-1stm for join order selection," in *2020 IEEE 36th international conference on data engineering (ICDE)*. IEEE, 2020, pp. 1297–1308.
- [5] R. Marcus and O. Papaemmanouil, "Deep reinforcement learning for join order enumeration," in *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*, ser. aiDM'18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3211954.3211957>
- [6] A. Cheng, A. Kabcenell, J. Chan, X. Shi, P. Bailis, N. Crooks, and I. Stoica, "Towards optimal transaction scheduling," *Proc. VLDB Endow.*, vol. 17, no. 11, p. 2694–2707, Jul. 2024. [Online]. Available: <https://doi.org/10.14778/3681954.3681956>
- [7] S. Chen, C. Shen, and C. Wu, "Intelligent transaction scheduling to enhance concurrency in high-contention workloads," *Applied Sciences*, vol. 15, no. 11, 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/15/11/6341>
- [8] T. K. Sellis, "Multiple-query optimization," *ACM Trans. Database Syst.*, vol. 13, no. 1, p. 23–52, Mar. 1988. [Online]. Available: <https://doi.org/10.1145/42201.42203>
- [9] T. Kathuria and S. Sudarshan, "Efficient and provable multi-query optimization," in *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, ser. PODS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 53–67. [Online]. Available: <https://doi.org/10.1145/3034786.3034792>
- [10] S. Agrawal, S. Chaudhuri, and V. R. Narasayya, "Automated selection of materialized views and indexes in sql databases," in *Proceedings of the 26th International Conference on Very Large Data Bases*, ser. VLDB '00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, p. 496–505.
- [11] S. Chaudhuri and V. R. Narasayya, "An efficient cost-driven index selection tool for microsoft sql server," in *Proceedings of the 23rd International Conference on Very Large Data Bases*, ser. VLDB '97. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997, p. 146–155.
- [12] W. Wu, C. Wang, T. Siddiqui, J. Wang, V. Narasayya, S. Chaudhuri, and P. A. Bernstein, "Budget-aware index tuning with reinforcement learning," in *Proceedings of the 2022 International Conference on Management of Data*, ser. SIGMOD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1528–1541. [Online]. Available: <https://doi.org/10.1145/3514221.3526128>
- [13] D. Shasha and P. Bonnet, *Database tuning: principles, experiments, and troubleshooting techniques*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002.
- [14] S. Cluet and G. Moerkotte, "On the complexity of generating optimal left-deep processing trees with cross products," in *International Conference on Database Theory*. Springer, 1995, pp. 54–67.
- [15] P. G. Selinger, M. M. Astrahan *et al.*, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD '79)*, 1979, p. 23–34. [Online]. Available: <https://doi.org/10.1145/582095.582099>
- [16] T. Ibaraki and T. Kameda, "On the optimal nesting order for computing n-relational joins," *ACM Trans. Database Syst.*, vol. 9, no. 3, p. 482–502, Sep. 1984. [Online]. Available: <https://doi.org/10.1145/1270.1498>
- [17] G. Moerkotte and T. Neumann, "Dynamic programming strikes back," in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, 2008, pp. 539–552.
- [18] T. Neumann and B. Radke, "Adaptive optimization of very large join queries," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 677–692.
- [19] I. Sabek, T. S. Ukyab, and T. Kraska, "LSched: A Workload-Aware Learned Query Scheduler for Analytical Database Systems," in *SIGMOD*, 2022.
- [20] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proc. VLDB Endow.*, vol. 12, no. 11, p. 1705–1718, jul 2019. [Online]. Available: <https://doi.org/10.14778/3342263.3342644>
- [21] "Techrepublic: D-wave announces 5,000-qubit fifth generation quantum annealer," <https://www.dwavequantum.com/company/newsroom/media-coverage/techrepublic-d-wave-announces-5-000-qubit-fifth-generation-quantum-annealer/>.
- [22] IBM Quantum, "Cloud access to quantum computers provided by ibm," <https://quantum-computing.ibm.com>, 2022.
- [23] Rigetti Computing, "Rigetti quantum processors," <https://qcs.rigetti.com/qpus>, 2022.
- [24] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*, 1st ed. USA: Cambridge University Press, 2009.

- [25] E. Bernstein and U. Vazirani, "Quantum complexity theory," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1411–1473, 1997.
- [26] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96. New York, NY, USA: Association for Computing Machinery, 1996, p. 212–219. [Online]. Available: <https://doi.org/10.1145/237814.237866>
- [27] Google, "Google hails breakthrough as quantum computer surpasses ability of supercomputers," <https://www.theguardian.com/technology/2025/oct/22/google-hails-breakthrough-as-quantum-computer-surpasses-ability-of-supercomputers>, 2025.
- [28] A. D. King, A. Nocera, M. M. Rams *et al.*, "Beyond-classical computation in quantum simulation," *Science*, vol. 388, no. 6743, pp. 199–204, 2025. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.ad6285>
- [29] Z. Liu, "Practical Quantum Advantage for Boosting Citations," 3 2025.
- [30] D. Boneh and R. J. Lipton, "Quantum cryptanalysis of hidden linear functions," in *Advances in Cryptology — CRYPTO' 95*, D. Coppersmith, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1995, pp. 424–437.
- [31] Y. Zhao, "Demonstration and implementation of quantum computing in cryptanalysis," vol. 38, p. 431–436, Mar. 2023. [Online]. Available: <https://drpress.org/ojs/index.php/HSET/article/view/5855>
- [32] A. Awasthi, N. Kraus *et al.*, "Real world application of quantum-classical optimization for production scheduling," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 02, 2024, pp. 239–244.
- [33] G. Colucci *et al.*, "Power network optimization: A quantum approach," *IEEE Access*, vol. 11, pp. 98 926–98 938, 2023.
- [34] D-Wave Leap's Hybrid Solvers, https://docs.dwavesys.com/docs/latest/doc_leap_hybrid.html, 2024.
- [35] Google, "Google: Meet willow, our state-of-the-art quantum chip," <https://blog.google/technology/research/google-willow-quantum-chip/#:~:text=General%20summary,medicine%2C%20energy%2C%20and%20AI,2024>.
- [36] Microsoft, "Microsoft's majorana 1 chip carves new path for quantum computing," <https://news.microsoft.com/source/features/innovation/microsofts-majorana-1-chip-carves-new-path-for-quantum-computing/>, 2025.
- [37] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, Aug. 2018. [Online]. Available: <https://doi.org/10.22331/q-2018-08-06-79>
- [38] M. Razavy, *Quantum theory of tunneling*. World Scientific, 2013.
- [39] D.-W. documentation, https://docs.dwavesys.com/docs/latest/c_gs_4.html, 2024.
- [40] K. Boothby, A. D. King, and J. Raymond, "Zephyr topology of d-wave quantum processors," in *D-Wave Technical Report*, 2021, p. 1–18.
- [41] R. Hai, S.-H. Hung, T. Coopmans, T. Littau, and F. Geerts, "Quantum data management in the nisq era," in *Proceedings of the International Conference on Very Large Data Bases, VLDB (To Appear)*, 2026.
- [42] T. Bittner and S. Groppe, "Avoiding blocking by scheduling transactions using quantum annealing," in *Proceedings of the 24th Symposium on International Database Engineering & Applications*, ser. IDEAS '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3410566.3410593>
- [43] S. Roy, L. Kot, and C. Koch, "Quantum databases," in *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org, 2013. [Online]. Available: http://cidrdb.org/cidr2013/Papers/CIDR13_Paper86.pdf
- [44] I. Trummer and C. Koch, "Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer," in *VLDB*, 2016.
- [45] M. Schönberger, I. Trummer, and W. Mauerer, "Quantum optimisation of general join trees," in *VLDB Workshops*, 2023.
- [46] M. Schönberger, S. Scherzinger *et al.*, "Ready to leap (by co-design)? join order optimisation on quantum hardware," *Proc. ACM Manag. Data*, vol. 1, no. 1, may 2023. [Online]. Available: <https://doi.org/10.1145/3588946>
- [47] S. Endo, Z. Cai, S. C. Benjamin, and X. Yuan, "Hybrid quantum-classical algorithms and quantum error mitigation," *Journal of the Physical Society of Japan*, vol. 90, no. 3, p. 032001, 2021.
- [48] Hybrid Solver for Constrained Quadratic Models [WhitePaper], https://www.dwavesys.com/media/rldh2ghw/14-1055a-a_hybrid_solver_for_constrained_quadratic_models.pdf, 2021.
- [49] H. Liu, F. Spedalieri, and I. Sabek, "A demonstration of q2o: Quantum-augmented query optimizer," in *Proceedings of the International Conference on Very Large Data Bases, VLDB*, 2025.
- [50] H. Liu, P. Saxena, F. Spedalieri, and I. Sabek, "Optimizing join orders via constrained quadratic models (abstract)," in *Proc. 1st Workshop Quantum Data and Machine Learning*, ser. QDML '25, 2025.
- [51] H. Liu, A. Kumar, F. Spedalieri, and I. Sabek, "Hybrid quantum-classical optimization for bushy join trees," in *Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications*, ser. Q-Data '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 20–24. [Online]. Available: <https://doi.org/10.1145/3736393.3736695>
- [52] E. Osaba and P. Miranda-Rodriguez, "D-wave's nonlinear-program hybrid solver: Description and performance analysis," *IEEE Access*, vol. 13, pp. 4724–4736, 2025.
- [53] M. Aramon, G. Rosenberg *et al.*, "Physics-inspired optimization for quadratic unconstrained problems using a digital annealer," *Frontiers in Physics*, vol. 7, p. 48, 2019.
- [54] M. Schönberger, I. Trummer, and W. Mauerer, "Quantum-Inspired Digital Annealing for Join Ordering," in *VLDB*, 2023.
- [55] N. Nayak, J. Rehfeld *et al.*, "Constructing optimal bushy join trees by solving qubo problems on quantum hardware and simulators," in *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments*, ser. BIDEDE '23, 2023. [Online]. Available: <https://doi.org/10.1145/3579142.3594298>
- [56] S. Groppe and J. Groppe, "Optimizing transaction schedules on universal quantum computers via code generation for grover's search algorithm," in *Proceedings of the 25th International Database Engineering & Applications Symposium*, ser. IDEAS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 149–156. [Online]. Available: <https://doi.org/10.1145/3472163.3472164>
- [57] N. Nayak, A. Prisacar, U. Çalkıylmaz, J. Groppe, and S. Groppe, "Quantum-enhanced transaction scheduling with reduced complexity via solving qubo iteratively using a locking mechanism," in *Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications*, ser. Q-Data '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 26–35. [Online]. Available: <https://doi.org/10.1145/3736393.3736701>
- [58] T. Fankhauser, M. E. Solèr, R. M. Füchslin, and K. Stockinger, "Multiple query optimization using a hybrid approach of classical and quantum computing," *arXiv preprint arXiv:2107.10508*, 2021.
- [59] C. Lee, C.-S. Shih, and Y.-H. Chen, "Optimizing large join queries using a graph-based approach," *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, no. 2, pp. 298–315, 2001.
- [60] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse ising model," *Phys. Rev. E*, vol. 58, pp. 5355–5363, Nov 1998. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevE.58.5355>
- [61] P. J. Van Laarhoven and E. H. Aarts, "Simulated annealing," in *Simulated annealing: Theory and applications*. Springer, 1987, pp. 7–15.
- [62] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, vol. 2, 2014. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fphy.2014.00005>
- [63] P. Saxena, I. Sabek, and F. Spedalieri, "Constrained quadratic model for optimizing join orders," in *Proc. 1st Workshop Quantum Comput. Quantum-Inspired Technol. Data-Intensive Syst. Appl.*, ser. Q-Data '24, 2024, p. 38–44. [Online]. Available: <https://doi-org.libproxy2.usc.edu/10.1145/3665225.3665447>
- [64] H. Karimi and G. Rosenberg, "Boosting quantum annealer performance via sample persistence," *Quantum Information Processing*, vol. 16, no. 7, p. 1–24, Jul. 2017. [Online]. Available: <https://doi.org/10.1007/s11128-017-1615-x>
- [65] "dimod documentation," <https://docs.ocean.dwavesys.com/en/stable/docsdimod/>.
- [66] Z. Bian, F. Chudak, R. Israel, B. Lackey, W. G. Macready, and A. Roy, "Discrete optimization using quantum annealing on sparse ising models," *Frontiers in Physics*, vol. 2, Sep. 2014. [Online]. Available: <https://doi.org/10.3389/fphy.2014.00056>
- [67] I. Trummer and D. Venturelli, "Leveraging quantum computing for database index selection," in *Proceedings of the 1st Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications*, ser. Q-Data '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 14–26. [Online]. Available: <https://doi.org/10.1145/3665225.3665445>

- [68] “qiskit-optimization documentation,” <https://qiskit.org/ecosystem/optimization/index.html>.
- [69] F. Limited, “Fujitsu digital annealer - solving large-scale combinatorial optimization problems instantly,” 2023, <https://www.fujitsu.com/global/services/business-services/digital-annealer/what-is-digital-annealer/>.
- [70] The D-Wave Advantage System [WhitePaper], https://www.dwavesys.com/media/3xvdipcn/14-1058a-a_advantage_processor_overview.pdf, 2022.
- [71] M. Schönberger, I. Trummer, and W. Mauerer, “Large-scale multiple query optimisation with incremental quantum(-inspired) annealing,” in *Proceedings of the ACM International Conference on Management of Data, SIGMOD (To Appear)*, 2026.
- [72] I. Haffner and J. Dittrich, “Efficiently computing join orders with heuristic search,” *Proc. ACM Manag. Data*, vol. 1, no. 1, May 2023. [Online]. Available: <https://doi.org/10.1145/3588927>
- [73] T. Zhang, A. Tomasic, and A. Pavlo, “Intelligent transaction scheduling via conflict prediction in oltp dbms,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.01675>
- [74] V. Sharma and C. Dyreson, “Indexer++: workload-aware online index tuning with transformers and reinforcement learning,” in *Proceedings of the 37th ACM SIGAPP Symposium on Applied Computing, ser. SAC '22*. New York, NY, USA: Association for Computing Machinery, 2022, p. 372–380. [Online]. Available: <https://doi.org/10.1145/3477314.3507691>
- [75] B. Lackey, “A belief propagation algorithm based on domain decomposition,” 2018. [Online]. Available: <https://arxiv.org/abs/1810.10005>
- [76] S. Kullback and R. A. Leibler, “On information and sufficiency,” *The annals of mathematical statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [77] Z. Bian, F. A. Chudak, R. B. Israel, B. Lackey, W. G. Macready, and A. Roy, “Mapping constrained optimization problems to quantum annealing with application to fault diagnosis,” *Frontiers ICT*, vol. 3, p. 14, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:7012467>
- [78] J. Lin, “Divergence measures based on the shannon entropy,” *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.
- [79] T. Noguchi, K. Fukada, S. Bao, and N. Togawa, “Subqubo annealing based on efficient binary variable selection for combinatorial optimization problems with one-hot constraints,” in *2025 IEEE International Conference on Consumer Electronics (ICCE)*, 2025, pp. 1–2.
- [80] W. Zhao and G. Tang, “Clustering-based sub-qubo extraction for hybrid qubo solvers,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.16212>
- [81] M. Booth, S. P. Reinhardt, and A. Roy, “Partitioning optimization problems for hybrid classical-quantum execution,” in *D-Wave Technical Report*, 2017, p. 1–9.
- [82] E. Zamanian, J. Shun, C. Binnig, and T. Kraska, “Chiller: Contention-centric transaction execution and data partitioning for modern networks,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, ser. SIGMOD '20*. New York, NY, USA: Association for Computing Machinery, 2020, p. 511–526. [Online]. Available: <https://doi.org/10.1145/3318464.3389724>
- [83] D. Pisinger and S. Ropke, *Large Neighborhood Search*. Cham: Springer International Publishing, 2019, pp. 99–127. [Online]. Available: https://doi.org/10.1007/978-3-319-91086-4_4
- [84] PostgreSQL, <https://www.postgresql.org/>.
- [85] PostgreSQL Global Development Group, <https://www.postgresql.org/>.
- [86] MySQL Documentation Team, *MySQL 8.4 Reference Manual*, 8th ed., 2025. [Online]. Available: <https://dev.mysql.com/doc/en/>
- [87] Oracle Corporation, *Oracle Database Reference, 19c*, 19th ed., Apr. 2025. [Online]. Available: <https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/index.html>
- [88] H. Liu, Q. Zhang, R. Marcus, and I. Sabek, “Serag: Self-evolving rag system for query optimization,” 2025. [Online]. Available: https://viterbi-web.usc.edu/~sabek/pdf/25_workshop_serag.pdf
- [89] —, “Sefrqo: A self-evolving fine-tuned rag-based query optimizer,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.17556>
- [90] OSSC-DB, “pg_hint_plan,” https://github.com/oss-db/pg_hint_plan.
- [91] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?” *Proc. VLDB Endow.*, vol. 9, no. 3, p. 204–215, nov 2015. [Online]. Available: <https://doi.org/10.14778/2850583.2850594>
- [92] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, and M. Alizadeh, “Flow-loss: Learning cardinality estimates that matter,”

Proc. VLDB Endow., vol. 14, no. 11, p. 2019–2032, jul 2021. [Online]. Available: <https://doi.org/10.14778/3476249.3476259>